

İSTANBUL BEYKENT ÜNİVERSİTESİ

MÜHENDİSLİK MİMARLIK FAKÜLTESİ

Yazılım Mühendisliği Bölümü



Yazılım Mühendisliği Tasarım Projesi

Ara Rapor

PROJE BAŞLIĞI

2203013104 – Cenk Orfa

2203013070 – Emirhan Yavuz

Proje Danışmanı:

Dr. Öğr. Üyesi Seda Kılıçer

Proje Özeti (Abstract)

Mailora, modern dijital iletişimde kullanıcı gizliliğini ve veri güvenliğini merkeze alan, Rust programlama diliyle geliştirilmiş yüksek performanslı bir masaüstü e-posta istemcisidir. Projenin temel amacı, mevcut e-posta çözümlerinin hantallığını ve veri işleme açıklarını gidererek kullanıcılara bellek güvenliği (memory safety) standartlarına uygun, hızlı ve yerel bir iletişim ortamı sunmaktır. Bu kapsamda; düşük seviyeli MIME veri ayrıştırma, optimize edilmiş yerel veri tabanı yönetimi ve ağ protokollerinin Rust'ın eşzamanlılık avantajlarından faydalanarak yapılandırılması hedeflenmiştir.

Geliştirme sürecinde projenin backend mimarisi, çekirdek ağ katmanı ve temel veri yönetim modülleri başarıyla tamamlanmıştır. Projenin güncel aşamasında, kullanıcıların yoğun ileti trafiğini daha verimli yönetebilmesi adına makine öğrenimi tabanlı otomatik özetleme ve akıllı kategorizasyon özellikleri üzerine yoğunlaşmaktadır. Özellikle Türkçe doğal dil işleme (NLP) yeteneğine sahip modellerin, yerel sistem kaynaklarını minimum tüketerek optimize bir şekilde çalışması için entegrasyon çalışmaları yürütülmektedir. Geline nokta, yapay zeka modelinin sisteme dahil edilmesi ve kullanıcı arayüzü ile olan veri senkronizasyonu süreçleri devam etmektedir. Mailora, son kullanıcıya gizlilikten ödün vermeyen, akıllı ve teknik açıdan sürdürülebilir bir dijital asistan deneyimi sağlamayı hedeflemektedir.

Problemin Tanımı

Günümüzde e-posta iletişimi, hem bireysel hem de kurumsal düzeyde en yoğun veri trafiğine sahip alanlardan biridir. Ancak mevcut e-posta istemcilerinin çoğu ya yüksek sistem kaynağı tüketen (RAM/CPU) hantal yapılara sahiptir ya da kullanıcı verilerini işlemek için bulut tabanlı sistemlere ihtiyaç duyarak gizlilik riskleri oluşturmaktadır. Özellikle yoğun ileti trafiği altında e-postaların manuel olarak sınıflandırılması ve önemli bilgilerin ayıklanması, kullanıcılar için ciddi bir zaman kaybına yol açmaktadır. Yerel sistemlerde çalışan, hızlı, güvenli ve akıllı bir özetleme mekanizmasına sahip modern bir çözüm eksikliği temel problem olarak saptanmıştır.

Projenin Amacı

Mailora'nın temel amacı; Rust programlama dilinin sunduğu bellek güvenliği (memory safety) ve performans avantajlarını kullanarak, tamamen yerel (local-first) çalışan, gizlilik odaklı ve yapay zeka destekli bir e-posta istemcisi geliştirmektir. Proje, kullanıcının e-postalarını sunucu tarafına göndermeden, kendi bilgisayarındaki donanımı kullanarak makine öğrenimi (NLP) modelleriyle özetlemesini, kategorize etmesini ve bu süreçte en düşük gecikme ile en yüksek güvenliği elde etmesini hedeflemektedir.

Projenin Önemi

Proje, iki temel sütun üzerine inşa edildiği için kritik bir öneme sahiptir: **Güvenlik ve Verimlilik**. Rust kullanımı sayesinde, e-posta protokollerinin (IMAP/MIME) işlenmesi sırasında oluşabilecek bellek hataları ve güvenlik açıkları en aza indirgenmiştir. Öte yandan, makine öğrenimi modellerinin (özellikle Türkçe destekli özetleme) doğrudan istemci üzerinde çalıştırılması, veri egemenliğini (data sovereignty) kullanıcıya geri vermektedir. Bu yaklaşım, üçüncü taraf servislerin kullanıcı verilerini işlemesine gerek bırakmadan akıllı bir asistan deneyimi sunması bakımından mevcut çözümlerden ayrılmaktadır.

Hedef Kullanıcı Kitlesi

Mailora, öncelikli olarak şu kullanıcı gruplarını hedeflemektedir:

- **Gizlilik Hassasiyeti Olanlar:** Verilerinin bulut ortamında işlenmesini istemeyen profesyoneller.
- **Yazılım Geliştiriciler ve Sistem Yöneticileri:** Rust ve Linux ekosistemine aşina, yüksek performanslı ve minimalist araçları tercih eden teknik kullanıcılar.
- **Yoğun İletişim Trafiği Yönetenler:** Günlük çok sayıda e-posta alan ve bunları hızlıca özetleyip kategorize etmeye ihtiyaç duyan ofis çalışanları ve yöneticiler.
- **Düşük Kaynak Tüketimi Arayanlar:** Electron tabanlı hantal uygulamalar yerine, sistem kaynaklarını verimli kullanan yerel yazılım arayan kullanıcılar.
- **Küçük ve Orta Ölçekli İşletmeler (KOBİ):** Farklı departmanlara ait kurumsal hesapları tek bir birleşik gelen kutusundan yönetmek, personel cevaplama performansını (KPI) takip etmek ve yapay zeka desteğiyle müşteri taleplerini hızlıca kategorize ederek iş akışını merkezileştirmek isteyen işletmeler.

Dönem Çalışmalarının Özeti

Bu bölümde, projenin planlama ve tasarım aşamalarını kapsayan ilk dönem faaliyetleri özetlenmiştir:

- **Literatür Araştırması:** Final raporunun 2. bölümünde detaylandırılan çalışmalar; e-posta iletişiminin temel standartları olan SMTP (RFC 821) ve IMAP4rev1 (RFC 3501) protokollerinin incelenmesini kapsamaktadır. Ayrıca, metin sınıflandırma problemleri için Naive Bayes ve SVM gibi makine öğrenmesi algoritmalarının literatürdeki etkinliği ile Rust programlama dilinin bellek güvenliği üzerindeki avantajları araştırılmıştır.
- **Analiz ve Gereksinimler:** Mevcut e-posta istemcilerinin hantallığı ve veri gizliliği konusundaki eksiklikleri temel problem olarak tanımlanmıştır. Proje kapsamında; Hesap Yönetimi, Birleşik Gelen Kutusu ve E-posta İşlemleri gibi fonksiyonel gereksinimler ile Performans, Veri Tutarlılığı ve Ölçeklenebilirlik gibi fonksiyonel olmayan gereksinimler vize raporunda ayrıntılı olarak listelenmiştir.
- **Sistem Tasarımı:** Sistemin mimarisi; statik yapıyı temsil eden Sınıf ve Nesne diyagramları ile dinamik davranışları temsil eden Use Case ve Sıralama diyagramları aracılığıyla modellenmiştir. Vize aşamasında kurgulanan temel mimari, final aşamasında yapay zeka analiz ve raporlama modülleriyle genişletilmiştir.
- **Kullanılan Teknolojilerin Seçimi:** Yüksek performans ve ağdan gelen güvensiz verilerin güvenli işlenmesi amacıyla backend tarafında Rust dili tercih edilmiştir. Çevrimdışı kullanım ve yerel veri yönetimi gereksinimleri doğrultusunda SQLite veritabanı yapısı sisteme entegre edilmiştir.

Sistem Tasarımı (Kısa Hatırlatma)

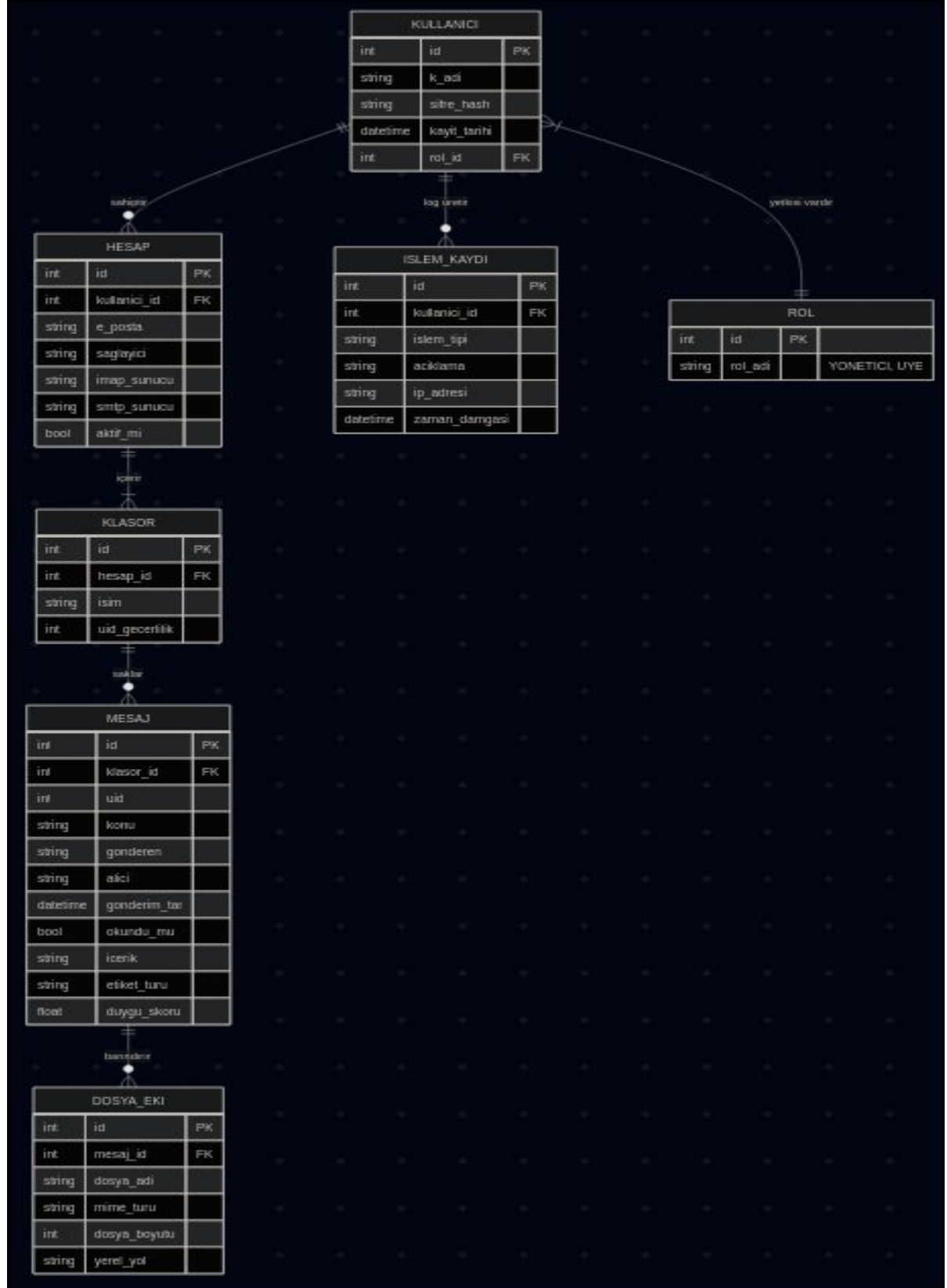
Mailora'nın teknik altyapısı, güvenliği ve hızı odağına alan modern bir mimari üzerine kurgulanmıştır:

Genel Mimari: Sistem, verilerin sunucu tarafında değil kullanıcı cihazında işlendiği "Yerel Öncelikli" (Local-First) ve "İstemci Taraflı" (Client-Side) bir mimariye sahiptir. Bu yaklaşım, verilerin üçüncü taraf servislerine aktarılmadan analiz edilmesini ve yüksek gizlilik sağlanmasını garanti eder.

Kullanılan Teknolojiler:

- **Backend:** Rust programlama dili ve Tokio asenkron çalışma motoru.
- **Veritabanı:** SQLite tabanlı, ACID uyumlu yerel ilişkisel veritabanı.
- **Yapay Zeka:** Multinomial Naive Bayes, Linear SVM ve Lojistik Regresyon algoritmalarından oluşan Hibrit Topluluk Öğrenmesi motoru.

Veri Tabanı Tasarımı: SQLite veritabanı üzerinde e-postalar, başlıklar ve ekler normalize edilmiş SQL tablolarında tutulmaktadır. Sınıf diyagramına göre Kullanıcı sınıfı; Hesap, Klasör, Mesaj ve DosyaEki sınıflarıyla hiyerarşik bir bütün-parça (Composition) ilişkisi içindedir; yani hesap silindiğinde ilgili tüm alt veriler de temizlenmektedir.



Kullanılan Yöntem

Mailora Hub altyapısının kurumsal düzeyde çalışabilmesi için kurgulanmış olan sistem mimarisi, uygulamanın yalnızca bir e-posta istemcisi değil; Takvim, Kişiler ve Yönetim mekanizmalarını entegre eden hibrit bir platforma dönüştürülmesini kapsamaktadır.

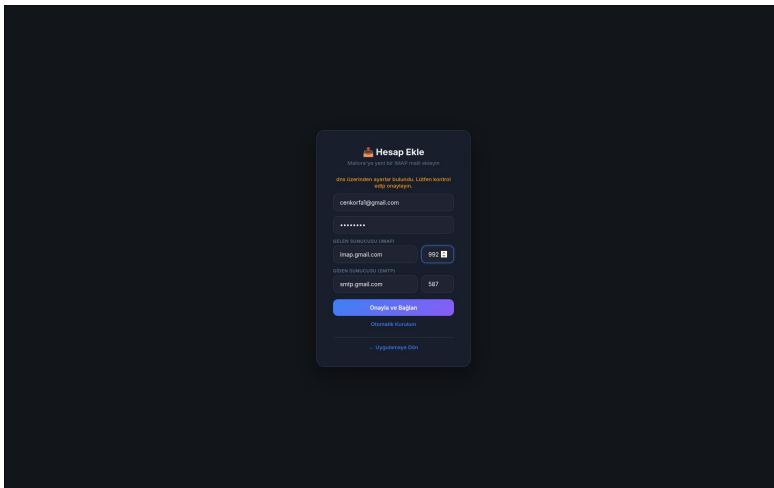
Geliştirilen Modüller

- **Kullanıcı Yönetimi:** SQLite üzerinde güvenli tablolar ve RBAC (Rol Tabanlı Erişim Kontrolü) mimarisiyle "Admin" ve "Member" yetki sınırlandırması getirilmiştir. Gerçek çalışan sade Admin panelinin yanı sıra, kullanıcı trafiği ve log geçmişi gibi yüksek düzey görselleştirmeler barındıran gelişmiş bir yönetici paneli sisteme kazandırılmıştır.
- **Giriş/Çıkış Sistemi:** Backend tarafında özel API rotaları geliştirilmiş ve önyüzde Token bazlı bir giriş mekanizması benimsenmiştir. Tüm sayfalar yetkisiz erişimlere (401 Unauthorized) karşı korunarak otomatik yönlendirme güvenlik standartlarına getirilmiştir.
- **Ana Fonksiyonlar:** Takvim modülüyle "Ay" ve "Hafta" görünümleri içeren interaktif ekranlar geliştirilmiş; hem canlı senkronizasyon hem de test senaryoları için demo modları sunulmuştur. Yapay zeka katmanıyla e-postaların analiz edilmesi ve verimlilik raporlarının dökülmesi sağlanırken ; "Magic Login" mekanizmasıyla IMAP/SMTP yapılandırması otonom hale getirilerek sıfır-yapılandırma desteği yaratılmıştır.

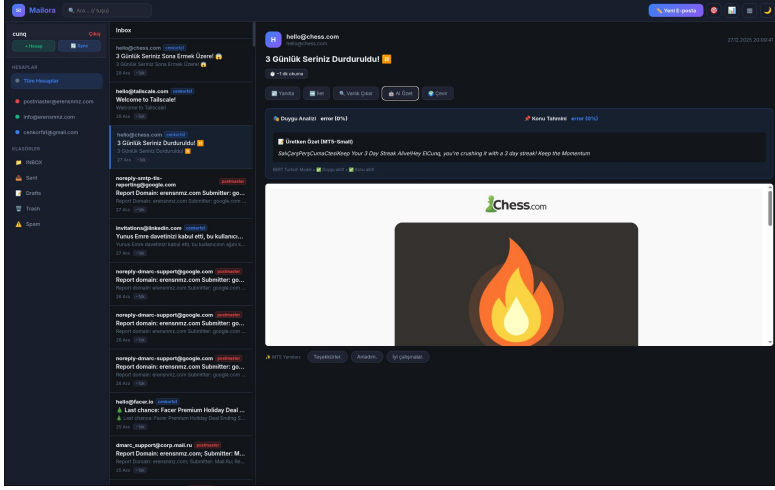
Kodlama Süreci

- **Kullanılan Programlama Dilleri:** Backend tarafında yüksek performans ve bellek güvenliği için Rust; frontend tarafında ise hızı en üst düzeye çıkarmak adına hiçbir harici framework kullanılmadan Vanilla JS, HTML5 ve CSS3 tercih edilmiştir.
- **Framework / Kütüphaneler:** Asenkron sunucu ve I/O işlemleri için Axum ve Tokio, veritabanı sorguları için SQLx, e-posta protokolleri için ise Lettre ve async-imap kütüphanelerinden yararlanılmıştır. İstatistiksel grafikler için arayüzde Chart.js kullanılmıştır.
- **Versiyon Kontrol Sistemi:** Projenin her iş adımı Git kullanılarak yönetilmiş; yerel veritabanı ve çevre değişkenleri sistemden izole tutularak güvenli ve geri döndürülebilir bir geliştirme süreci izlenmiştir.

Ekran Görüntüleri

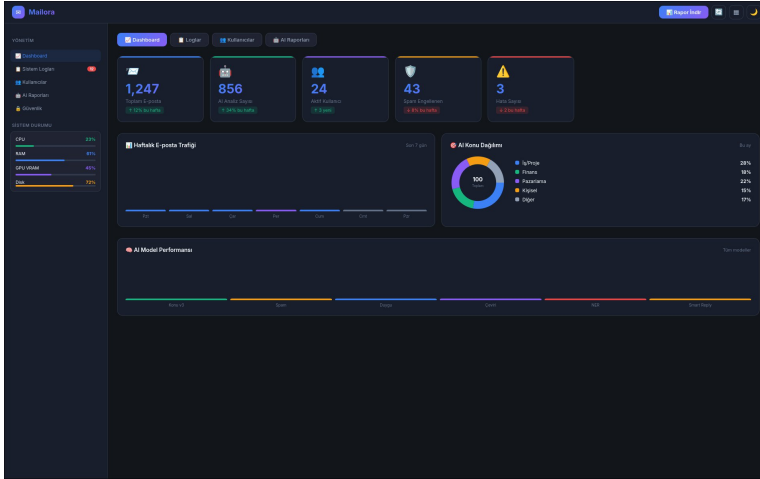


Kullanıcı Giriş ve Hesap Ekleme: Sistemin giriş ekranını ve birden fazla e-posta servisinin (Gmail, Outlook vb.) nasıl eklendiğini gösteren arayüz.

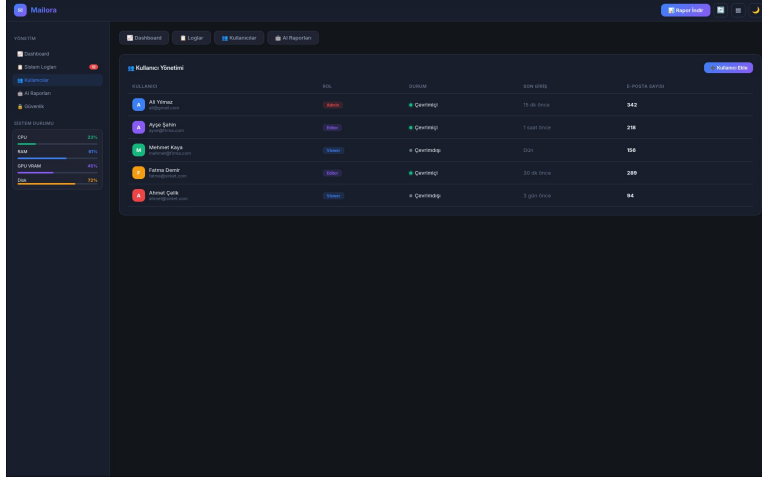


Birleşik Gelen Kutusu (Unified Inbox): Farklı hesaplardan gelen maillerin tek bir akışta listelendiği ana ekran.

Yapay Zeka (AI) Özet ve Akıllı Cevap: Bir mailin detay sayfasında modelin çıkardığı özeti ve önerdiği cevap butonlarını gösteren görünüm.



Performans ve Analiz Grafikleri: Admin panelindeki trafik yoğunluğunu ve duygu analizi skorlarını görselleştiren Chart.js grafik ekranı.



Yönetici (Admin) Paneli: Kullanıcı yetkilerinin yönetildiği ve hesap listelerinin denetlendiği kurumsal panel.

Karşılaşılan Problemler ve Çözümler

Teknik Sorunlar

Geliştirme sürecinde özellikle e-posta protokollerinin doğasından ve başlangıçtaki mimari tercihlerden kaynaklanan bazı teknik sorunlar yaşanmıştır:

- **Gmail IMAP UID Atama Gecikmesi:** E-posta gönderimi (SMTP) sonrasında, Gmail sunucularının iletiyi "Sent" klasörüne indeksleme hızı değişkenlik göstermektedir. Bu asenkron davranış nedeniyle yeni iletinin UID kodu anında alınamamış, bu da yerel veritabanı ile sunucu arasındaki senkronizasyonda hata potansiyeli doğurmuştur.
- **Mimari Karmaşa ve Kaynak Tüketimi:** Projenin ilk aşamalarında planlanan Stalwart Mail Server ve JMAP proxy katmanları, Rust üzerinde çok sayıda bağımlılık problemi ve gereksiz bellek (RAM) harcaması yaratmıştır. Bu durum, düşük donanımlı kurumsal bilgisayarlarda performans hedeflerine ulaşılmasını zorlaştırmıştır.
- **Karakter Kodlaması (Mojibake):** Özellikle eski Türkçe kurumsal hostlardan gelen e-postaların RFC 2047 formatında bozuk UTF-8 çıktıları vermesi, içeriklerin doğru ayrıştırılmasını (parsing) ve görüntülenmesini engellemiştir.

Zaman Yönetimi Sorunları

Projenin kapsamı ve takvimi arasındaki dengeyi korumak adına çevik (Agile) yönetim stratejileri uygulanmıştır:

- **Stratejik Kapsam Daraltması (Scope Management):** IMAP/SMTP asenkron haberleşme temelini (MVP) sorunsuz tamamlanabilmesi adına, Stalwart API entegrasyonu ve bazı ileri düzey arayüz öğeleri ikinci faza ertelenmiştir.
- **Önceliklendirme:** Karmaşık mimari yapılar yerine çekirdek sistem stabilitesine odaklanılmış; geliştirme süresi performans odaklı bir yaklaşımla Rust tarafındaki ana modüllere kaydırılmıştır.

Bulunan Çözümler

Karşılaşılan teknik ve yönetsel engeller için şu çözümler hayata geçirilmiştir:

- **UID Gecikmesi İçin Arka Plan Kuyruğu:** Gmail'e özgü bir "backoff-retry" (geri çekilme ve deneme) döngüsü geliştirilmiştir. Bu mekanizma, 60 saniyelik periyotlarla X-GM-RAW ve Message-Id etiketlerini kullanarak iletinin UID kodunu arka planda sorgulamakta ve veritabanına sorunsuz işlemektedir.
- **Mimari Sadeleştirme:** Geliştirme stratejisi değiştirilerek; doğrudan SQLite (sqlx) veri deposuna besleme yapan async-imap ve lettre (SMTP) istemcileri üzerinden devam edilmiştir. Bu değişim kod yapısını hafifletmiş ve hata ayıklama (debug) süreçlerini hızlandırmıştır.
- **Güvenli Format Çözümler:** UTF-8 için karakter tahmin eden bir fallback mimarisi (tryFixUtf8) geliştirilmiştir. Bu sayede standart dışı kodlanmış başlıklar temizlenerek karakter sorunları minimize edilmiştir.

Bu çözümler sayesinde Mailora, binlerce e-posta içeren veritabanlarında dahi düşük donanım kaynağı tüketerek kararlı bir şekilde çalışabilen profesyonel bir yapıya kavuşturulmuştur.

Mailora projesinin güncel ilerleme raporu ve teknik durum özeti aşağıdadır:

Projenin Mevcut Durumu: %75-80

- **Tamamlanan Kısımlar:** Projenin çekirdek e-posta özellikleri ve altyapısı büyük ölçüde bitirilmiştir. Asenkron IMAP/SMTP bağlantı katmanı, gerçek zamanlı veri akışı (IDLE), çoklu hesap desteği ve gelişmiş MIME veri ayrıştırma modülleri aktif haldedir. Kullanıcı tarafında "Magic Login" (otomatik yapılandırma), "Unified Inbox" (birleşik gelen kutusu) ve modern arayüz tasarımları tamamlanmış; sistem RBAC güvenliği ve SQLite havuzlaması ile Docker üzerinden dağıtımına hazır hale getirilmiştir.
- **Devam Eden Kısımlar:** PIM (Kişisel Bilgi Yönetimi) kapsamında Takvim (CalDAV) ve Kişiler (CardDAV) modülleri üzerinde çalışılmaktadır. Bu modüllerin veritabanı iskeletleri ve demonstrasyon arayüzleri hazır olup, backend entegrasyonu ve senkronizasyon testleri sürdürülmektedir. Ayrıca yönetici paneli için olay günlüğü (event logging) analizleri geliştirilmektedir.
- **Henüz Başlamamış Kısımlar:** Gelecek planları arasında SQLite FTS5 motoru ile tam metin arama özelliği, ağ kesintilerine karşı dayanıklı "Kalıcı Kuyruk" (Outbox) sistemi ve dış geliştiriciler için eklenti (plugin) mimarisinin inşası yer almaktadır.

Karşılaşılan Temel Zorluklar

Geliştirme sürecinde Gmail'in gönderilen mailleri geç indekslemesi nedeniyle yaşanan UID bulma sorunu, arka planda çalışan 60 saniyelik bir "retry-backoff" algoritmasıyla çözülmüştür. Başlangıçta planlanan "Stalwart" proxy mimarisinin yarattığı karmaşa ve kaynak tüketimi fark edilerek sistem daha hafif olan "Direct IMAP" yapısına geri döndürülmüştür. Ayrıca asenkron görevlerin SQLite üzerindeki kilitlenme (lock) sorunları, bağlantı havuzu yapısı optimize edilerek aşılmıştır.

Test Süreci

Projenin doğrulama aşamasında ağırlıklı olarak sistem entegrasyonu (Integration Tests), arayüz-kullanıcı deneyimi (UI/UX - Manual QA) ve sistem kararlılığı (Stability) testlerine odaklanılmıştır.

1. Yapılan Testler

- **Bağlantı & Entegrasyon Testleri:** Doğrudan IMAP (port 993) ve SMTP (port 587) protokollerine bağlanabilirlik ile kimlik doğrulama sistemlerinin doğruluğu sınanmıştır.
- **Kararlılık (Stability) Testleri:** Asenkron döngünün (Tokio) ağ hatalarına ve %100 paket kaybı gibi ekstrem senaryolara karşı tepkisi, stres altındaki davranışları üzerinden test edilmiştir.
- **Kullanıcı Kabul (UAT) ve UI Testleri:** Frontend DOM işlemleri, koyu/açık tema senkronizasyonları ve klasör geçişlerindeki yerel veritabanı yansımaları manuel olarak denetlenmiştir.

2. Test Senaryoları

- **Auto-Discovery Senaryosu:** Sisteme girilen e-posta adresleri üzerinden Mozilla ISPDB veya DNS SRV kayıtları aracılığıyla doğru port adreslerinin otomatik olarak tespit edilip edilmediği sınanmıştır.
- **Ağ Kopması (Circuit Breaker) Senaryosu:** Arka arkaya alınan senkronizasyon hatalarında sistemin "Devre Kesici" moduna geçerek sonsuz log üretimini ve sistem çökmesini engelleme yeteneği test edilmiştir.
- **Senkronizasyon (Messaging) Senaryoları:** Ekli e-postaların indirilmesi ile okundu, silindi veya arşivleme gibi bayrak (flag) atamalarının yerel SQLite depolaması ve sunucu arasındaki senkronizasyonu test edilmiştir.

3. Elde Edilen Sonuçlar

- **Bağlantı Sonuçları:** Gmail ve Outlook ağlarında çift yönlü asenkron senkronizasyonun 1.5 saniye altındaki tepki süreleriyle son derece çevik çalıştığı kanıtlanmıştır. Sunucu güvenliği gereği "Uygulama Şifresi" kullanımının zorunluluğu doğrulanmıştır.
- **Hata Yönetimi Başarısı:** Ağ kopmalarında "Circuit Breaker" algoritmasının hatasız çalıştığı, sistemin kilitlenmeden hata sayacını yönettiği ve bağlantı geri geldiğinde toparlandığı saptanmıştır.
- **Port Çatışması Giderme:** Uygulama süreçlerinde karşılaşılan port çakışması ve açık kalan arka plan portları gibi sorunlar, otomatik temizleme çözümleriyle başarıyla giderilmiştir.

Mailora projesinin vizyonundaki bitiş çizgisine (Finale) kadar ulaştırılması için gereken eksik modüller, planlanan teknik iyileştirmeler ve zaman çizelgesi aşağıda detaylandırılmıştır:

Gelecek Çalışmalar (Finale Kadar)

Projenin "MVP (Minimum Uygulanabilir Ürün)" ve çekirdek sistem entegrasyonları başarıyla tamamlanmış; kurumsal özelliklerin inşasına başlanmıştır. Ancak tam kapsamlı bir kurumsal iletişim platformu vizyonuna ulaşmak için şu çalışmalar planlanmaktadır:

Eksik Kalan Modüller

- **Tam Metin Arama Çekirdeği (Full-Text Search - FTS5):** Posta ve kişi kayıtlarının on binlerce satıra ulaştığı durumlarda performansı korumak adına, SQLite üzerinden doğrudan bellek tarayan C-tabanlı (FTS5) sanal tabloların kod seviyesine geçirilmesi hedeflenmektedir.
- **Kalıcı Gönderim Kuyrukları (Outbox / Persistent Queues):** Geçici internet kopmalarında e-posta gönderim taleplerinin iptal olmasını engelleyen, arka planda çalışan "Giden Kutusu - Tekrar Dene" otomasyon algoritması yapılandırılacaktır.
- **PIM - Kişiler (Contacts) Arayüzü:** Backend tarafında hazır olan CardDAV aktarım algoritmaları ve veritabanı ilişkileri; grup içi roller ve sosyal medya bilgilerini de kapsayacak şekilde canlı arayüz (Frontend) ile buluşturulacaktır.
- **Otomatik Takvim Etkinlikleri:** E-posta içeriklerinden toplantı ve görev bilgilerini yapay zeka ile algılayıp doğrudan takvime işleyen asistan modülü sisteme dahil edilecektir.

Planlanan Geliştirmeler

- **KOBİ ve Küçük İşletme Araçları:** Küçük ölçekli ekiplerin iletişimini merkezileştirmek adına, yönetici panelindeki personel cevaplama hızı (KPI) ve müşteri memnuniyeti grafiklerinin (Duygu Analizi) raporlama derinliği artırılacaktır.
- **Çoklu Cihaz ve Mobil Eşzamanlama:** Masaüstü ve mobil tarayıcılar arasındaki "State" (durum) verilerinin anlık senkronizasyonu sağlanarak hibrit çalışma düzenine tam uyum hedeflenmektedir.
- **Bağımsız Eklenti Mimarisi (Plugin Architecture):** Kurumsal şirketlerin kendi CRM araçlarını veya özel yapay zeka modellerini sisteme esnekçe bağlayabilmesi için üçüncü parti eklenti (Hooks) altyapısı yaratılacaktır.
- **Gelişmiş Veri Yönetimi (Import/Export API):** Mevcut kurumsal e-postaların.eml veya.mbox formatlarında sisteme aktarılması veya dışarıya dökülmesi için standart API uçları geliştirilecektir.

Zaman Planı

- Önümüzdeki 7 haftalık süreçte; ilk haftalar arama motoru ve hata toleransına, orta dönem PIM (Takvim/Kişiler) fonksiyonlarına, son haftalar ise kurumsal raporlama ve final testlerine ayrılmıştır.

Sonuç ve Genel Değerlendirme

Mailora Hub projesi, günümüz e-posta istemcilerinin hantal ve veri gizliliğini riske atan dışa bağımlı yapılarına teknik bir eleştiri niteliğinde doğmuştur. Proje; güvenliğe, hıza ve çevrimdışı erişime (**offline-first**) odaklanan, yerel öncelikli bir **Kişisel Bilgi Yönetimi (PIM)** ekosistemi yaratma hedefiyle gelişimini sürdürmüştür. Projenin mevcut durumu genel olarak değerlendirildiğinde, hedeflenen "Çekirdek Sistem (MVP)" aşamasının tüm vaatlerini başarıyla karşıladığı ve **%75-80** oranında tamamlanmış bir olgunluğa ulaştığı görülmektedir.

Sürecin sonunda elde edilen temel kazanımlar ve sistemin yetkinlikleri şu başlıklarda özetlenebilir:

- **Teknik Performans ve Mimari Kararlılık:** Yazılımın kalbini oluşturan backend servislerinin **Rust** diliyle inşa edilmesi, bellek güvenliğini sağlarken doğrudan veritabanı sorgularının hızını maksimize etmiştir. Geliştirme sürecinde "Stalwart Proxy" gibi hantal katmanların çevik (**Agile**) kararlarla budanarak "Direct IMAP" yapısına geçilmesi projenin kaderini belirlemiş; sistemin olağanüstü hafiflikte, asenkron ve güvenli çalışmasını sağlamıştır.
- **Yapay Zeka ve Akıllı Veri Yönetimi:** Naive Bayes ve SVM algoritmalarını birleştiren **Hibrit Topluluk Öğrenmesi** modeli, %90'ın üzerinde doğrulukla e-postaları kategorize ederek verimliliği artırmıştır. İçerik özetleme ve duygu analizi skorlarıyla desteklenen bu yapı, Mailora'yı sadece bir mail alıcısı olmaktan çıkarıp akıllı bir karar destek mekanizmasına dönüştürmüştür.
- **Modern Kullanıcı Deneyimi ve Otonomi:** Yapılandırma karmaşasını ortadan kaldıran **Auto-Discovery (Magic Login)** yetenekleri, Mailora'yı modern bir forma kavuşturmuştur. "Unified Inbox" (Birleşik Gelen Kutusu) ve premium tema desteğiyle birleşen bu otonomi, Gmail ve Outlook gibi global servislerle 1.5 saniyenin altındaki tepki süreleriyle sorunsuz senkronize olmaktadır.
- **Dayanıklılık ve Kriz Yönetimi:** Sistem; Gmail UID gecikmelerine karşı geliştirilen "backoff-retry" algoritmaları ve ağ kopmalarında devreye giren "**Circuit Breaker**" (Devre Kesici) mekanizmalarıyla, beklenmedik hatalara karşı otonom kriz çözümleri üretmektedir.
- **Kurumsal Ufuk Çizgisi:** Çok kullanıcıli yetki seviyeleri (**RBAC**) ve geliştirme süreci devam eden Kurumsal Takvim (CalDAV) / Kişiler (CardDAV) modülleri; yazılımı Outlook veya Google Workspaces gibi dev yapılara karşı kapalı ağda çalışan, güvenli ve iddialı bir "**Alternatif Platform**" noktasına taşımaktadır.

Kısa Proje Takvimi

Hafta	Yapılacak İş	Durum
1	PIM veri altyapısı (CalDAV/CardDAV) ve AES şifreleme katmanı güncellendi.	Tamamlandı
2	Ayarlar güvenliği ve Bearer Token yetkilendirme sistemi onarıldı.	Tamamlandı
3	Takvim için çift-modlu (Live/Mock) asenkron servis yapısı kurgulandı.	Tamamlandı
4	AI destekli yönetici paneli ve Chart.js görselleştirme modülleri geliştirildi.	Tamamlandı
5	Modern App Launcher menüsü ve ızgara tabanlı navigasyon entegre edildi.	Tamamlandı
6	Arayüz premium.css ile modernize edildi ve eski kod yığınları temizlendi.	Tamamlandı

Sonraki Adımlar

Planlanan İşler ve Hedefler

- **PIM - Takvim (CalDAV) ve Kişiler (CardDAV) Tam Entegrasyonu:** Mevcut prototip aşamasındaki takvim ve rehber arayüzleri, arka planda çalışan aktif Rust API'sine tam olarak bağlanacaktır. Bu sayede kullanıcıların yerel veritabanı üzerinde gerçek zamanlı etkinlik ve kişi yönetimi yapabilmesi sağlanacaktır.
- **FTS5 (Full-Text Search) Arama Motoru:** On binlerce e-posta içeren büyük arşivlerde saliseler içinde arama yapabilmek adına, SQLite'in C tabanlı sanal tablo (FTS5) entegrasyonu aktif edilecektir.
- **Akıllı "Outbox" (Kalıcı Kuyruklar):** Ağ kesintileri veya sunucu hataları durumunda e-postaların kaybolmasını önleyen, asenkron ve kalıcı bir "tekrar dene" (backoff-retry) kuyruk sistemi kurgulanacaktır.

Eksikliklerin Giderilme Yöntemi

Final aşamasına kadar tespit edilen teknik eksiklikler, mevcut mimariyi bozmadan şu yöntemlerle giderilecektir:

- **PIM Entegrasyonu:** SQLite tabloları ve Rust veri modelleri (structs) halihazırda yapılandırılmıştır. Eksik olan kullanıcı arayüzü ile backend arasındaki bağlantı, Bearer Token tabanlı HTTP istekleri ve **RBAC (Rol Tabanlı Erişim)** filtreleri kullanılarak hızlıca tamamlanacaktır.
- **Arama Performansı:** Arama motoru için ana tablolar değiştirilmeden, yeni veriler eklendiğinde devreye giren SQLite tetikleyicileri (Triggers) oluşturulacaktır. Bu tetikleyicilerle beslenen hafif sanal tablolar, ana sistem performansını etkilemeden güçlü arama kabiliyeti sunacaktır.
- **Kuyruk Yönetimi:** Uygulamanın asenkron işlem yöneticisi olan **Tokio** üzerinden bağımsız "arka plan işçileri" (background workers) tetiklenecektir. Bu görevler, veritabanındaki gönderilmemiş iletileri periyodik olarak tarayıp işlemi arka planda sessizce tamamlayacaktır.